

학생 성적 분석 프로그램

학생 이름과 점수를 입력받아 전체 성적을 분석하는 프로그램을 작성하시오.

이 프로그램은 학생별 합격 여부를 출력하고, 평균 점수, 최고 점수, 최저 점수, 합격자 목록, 점수 높은 순 정렬 결과를 출력해야 한다.

문제 설명

사용자에게 학생 수를 입력받는다.

그다음 학생 수만큼 학생 이름과 점수를 입력받는다. 입력받은 학생 정보는 리스트 안에 딕셔너리 형태로 저장한다.

점수는 반드시 0점 이상 100점 이하 여야 한다. 범위를 벗어난 점수를 입력하면 해당 학생의 점수를 다시 입력받아야 한다.

모든 입력이 끝나면 전체 학생 목록을 번호와 함께 출력한다. 점수가 60점 이상 이면 합격 , 60점 미만 이면 불합격 으로 표시한다.

이후 평균 점수, 최고 점수, 최저 점수, 합격자 목록을 출력하고, 마지막에는 점수가 높은 학생부터 정렬한 결과를 출력한다.

코드에서 사용할 변수명

다음 변수명을 사용해서 프로그램을 작성하시오.

```
student_count    # 입력받을 학생 수
students         # 학생 정보를 저장할 리스트
student_name     # 학생 이름
student_score    # 학생 점수
total_score      # 전체 점수 합계
average_score    # 평균 점수
max_score        # 최고 점수
min_score        # 최저 점수
passed_students  # 합격자 이름 목록
sorted_students  # 점수 높은 순으로 정렬된 학생 목록
```

코드에서 사용할 함수명

다음 함수를 정의해서 사용하시오.

```
input_students()
calculate_average_score(students)
find_max_score(students)
find_min_score(students)
find_passed_students(students)
print_students(students)
print_score_summary(students, average_score, max_score, min_score, passed_students)
print_sorted_students(sorted_students)
```

함수별 역할

`input_students()`

학생 수, 학생 이름, 학생 점수를 입력받는 함수이다.

점수가 0~100 범위를 벗어나면 다시 입력받아야 한다.

반환 예 :

```
return students
```

calculate_average_score(students)

학생 목록을 전달받아 평균 점수를 계산하고 반환하는 함수이다.

반환 예 :

```
return average_score
```

find_max_socre(students)

학생 목록에서 최고 점수를 찾아 반환하는 함수이다.

반환 예 :

```
return max_score
```

find_min_score(students)

학생 목록에서 최저 점수를 찾아 반환하는 함수이다.

반환 예 :

```
return min socre
```

find_passed_students(students)

점수가 60점 이상 인 학생 이름만 리스트로 만들어 반환하는 함수이다.

반환 예 :

```
return passed_students
```

print_students(students)

전체 학생 목록을 번호와 함께 출력하는 함수이다.

각 학생은 당므 형식으로 출력한다.

```
1. 김사과 - 90점 - 합격
```

```
print_score_summary(...)
```

평균 점수, 최고 점수, 최저 점수, 합격자 목록을 출력하는 함수이다.

```
print_sorted_students(sorted_students)
```

점수가 높은 순서대로 정렬된 학생 목록을 출력하는 함수이다.

입력 조건

- 학생 수는 정수로 입력받는다.
- 학생 이름은 문자열로 입력받는다.
- 학생 점수는 정수로 입력받는다.
- 점수는 0이상 100이하 만 허용한다.
- 잘못된 점수를 입력하면 다시 입력받는다.

출력 조건

프로그램은 다음 내용을 출력해야 한다.

- 전체 학생 목록
- 각 학생의 합격 여부
- 평균 점수
- 최고 점수
- 최저 점수
- 합격자 목록
- 점수 높은 순 정렬 결과

평균 점수는 소수점 첫째 자리까지 출력한다.

데이터 저장 조건

학생 정보는 반드시 다음과 같은 형태로 저장한다.

```
students = [  
    {"name": "김사과", "score": 90},  
    {"name": "반하나", "score": 75},  
    {"name": "오렌지", "score": 45}  
]
```

합격 기준

60점 이상: 합격
60점 미만: 불합격

입력 예시

```
학생 수: 3
학생 이름: 김사과
학생 점수: 90
학생 이름: 반하나
학생 점수: 75
학생 이름: 오렌지
학생 점수: 45
```

출력 예시

```
===== 전체 학생 목록 =====
1. 김사과 - 90점 - 합격
2. 반하나 - 75점 - 합격
3. 오렌지 - 45점 - 불합격

===== 성적 요약 =====
평균 점수: 70.0
최고 점수: 90
최저 점수: 45
합격자: 김사과, 반하나

===== 점수 높은 순 =====
1. 김사과 - 90점
2. 반하나 - 75점
3. 오렌지 - 45점
```

잘못된 점수 입력 예시

```
학생 수: 2
학생 이름: 김사과
학생 점수: 120
잘못된 점수입니다. 0~100 사이로 다시 입력하세요.
학생 점수: 90
학생 이름: 반하나
학생 점수: -10
잘못된 점수입니다. 0~100 사이로 다시 입력하세요.
학생 점수: 75
```

구현 조건

- `students` 는 리스트로 만든다.
- 각 학생 정보는 딕셔너리로 저장한다.
- 딕셔너리의 key는 `"name"`, `"score"` 를 사용한다.
- 학생 입력에는 `while`문 을 사용한다.
- 전체 학생 출력에는 `for`문 과 `enumerate()` 를 사용한다.

- 합격자 목록은 리스트 컴프리헨션으로 만든다.
- 점수 높은 순 정렬에는 `sorted()` 를 사용한다.
- 출력에는 f-string을 사용한다.
- 함수는 3개 이상 사용한다.
- 전역 변수에 의존하지 말고, 함수의 매개변수와 반환값을 사용한다.

메인 실행 흐름 예시

```
students = input_students()

average_score = calculate_average_score(students)
max_score = find_max_score(students)
min_score = find_min_score(students)
passed_students = find_passed_students(students)
sorted_students = sorted(students, key=lambda student: student["score"], reverse=True)

print_students(students)
print_score_summary(students, average_score, max_score, min_score, passed_students)
print_sorted_students(sorted_students)
```

실행 예제

```
학생 수: 3
학생 이름: 김사과
학생 점수: 90
학생 이름: 반하나
학생 점수: 75
학생 이름: 오렌지
학생 점수: 45

===== 전체 학생 목록 =====
1. 김사과 - 90점 - 합격
2. 반하나 - 75점 - 합격
3. 오렌지 - 45점 - 불합격

===== 성적 요약 =====
평균 점수: 70.0
최고 점수: 90
최저 점수: 45
합격자: 김사과, 반하나

===== 점수 높은 순 =====
1. 김사과 - 90점
2. 반하나 - 75점
3. 오렌지 - 45점
```